OPERATING SYSTEMS AND SYSTEM PROGRAMMING PRACTICAL – WEEK 4

Name: S Svara Haasini

Roll No: 2520090170

Task 1: wait() and waitpid()

Question:

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Source Code:

```
haasini@SSH:~/OSSP_upload/Practical_Week4$ cat > P4_T1.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void create_children_for_wait(void)
{
    printf("\n=== Demonstration using wait() ===\n");

    for (int i = 1; i <= 3; i++)
    {
        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork");
            exit(1);
        }

        if (pid == 0)
        {
            printf("Child %d created | PID = %d | Parent PID = %d\n",
                   i, getpid(), getppid());

            sleep(4 - i);

            printf("Child %d (PID %d) completed.\n",
EOF return 0;aitpid() can wait for a specific child PID.\n");hed_pid);
haasini@SSH:~/OSSP_upload/Practical_Week4$ gcc -Wall -Wextra P4_T1.c -o P4_T
1
```

Output:

```
haasini@SSH:~/OSSP_upload/Practical_Week4$ ./P4_T1
Parent Process PID = 700

=== Demonstration using wait() ===
Child 1 created | PID = 701 | Parent PID = 700
Child 2 created | PID = 702 | Parent PID = 700
Child 3 created | PID = 703 | Parent PID = 700
Child 3 (PID 703) completed.
wait() collected child PID = 703 | Exit status = 3
Child 2 (PID 702) completed.
wait() collected child PID = 702 | Exit status = 2
Child 1 (PID 701) completed.
wait() collected child PID = 701 | Exit status = 1

=== Demonstration using waitpid() ===
Child 1 created | PID = 704 | Parent PID = 700
Child 2 created | PID = 705 | Parent PID = 700
Parent will specifically wait for Child 2 PID = 705
Child 3 created | PID = 706 | Parent PID = 700
Child 3 (PID 706) completed.
Child 2 (PID 705) completed.
waitpid() specifically collected PID = 705 | Exit status = 2
Child 1 (PID 704) completed.
Remaining child PID 704 collected.
Remaining child PID 706 collected.

Comparison:
wait() waits for any terminated child.
waitpid() can wait for a specific child PID.
```

Observations:

• fork() creates multiple child processes. Each child receives its own PID.

• In the wait() demonstration, the parent creates three child processes (PIDs 701, 702, 703). The parent uses wait() to collect terminated children in the order they finish.

• Child 3 (PID 703) terminates first, and wait() collects it with exit status 3. Child 2 (PID 702) terminates next with exit status 2, and Child 1 (PID 701) terminates last with exit status 1.

• wait() returns the PID of any child process that has terminated, regardless of which specific child finished first. The order of collection depends on which child exits first.

• In the waitpid() demonstration, the parent creates three new children (PIDs 704, 705, 706). The parent explicitly specifies that it will wait for Child 2 (PID 705).

• waitpid(pid, ...) waits for a specific child PID. Only when Child 2 (PID 705) terminates does waitpid() return. The other children (PIDs 704, 706) are collected afterward.

• The key difference: wait() is useful when any child can be collected, and waitpid() provides greater control by allowing synchronization with a specific child PID.

Task 2: Zombie Process Investigation and Elimination

Question:

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

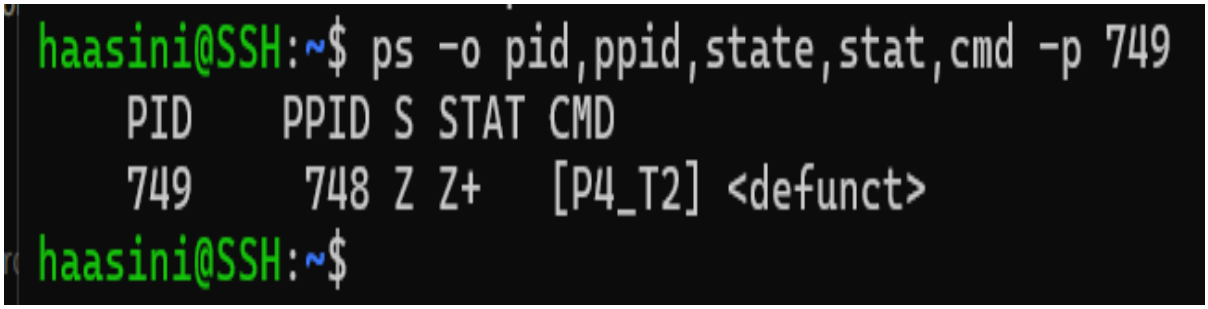
Source Code:

```
waitpid() can wait for a specific child PID.
haasini@SSH:~/OSSP_upload/Practical_Week4$ cat > P4_T2.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void create_zombie(void)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child PID = %d\n", getpid());
        printf("Child is terminating now.\n");
        exit(0);
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Child PID  = %d\n", pid);
        printf("Parent will NOT call wait() for 30 seconds.\n");
        printf("During this time the terminated child becomes a zombie.\n");
        printf("Use another terminal and run:\n");
        printf("ps -o pid,ppid,state,stat,cmd -p %d\n", pid);

        sleep(30);

        printf("Parent now calls waitpid() to collect child.\n");

        waitpid(pid, NULL, 0);

        printf("Zombie child has been collected.\n");
EOF return 0;n 1;nvalid option.\n");) == 0)y.\n"); using waitpid().\n");
```

Zombie Process Creation and Execution:

```
haasini@SSH:~/OSSP_upload/Practical_Week4$ ./P4_T2 zombie
Parent PID = 748
Child PID  = 749
Parent will NOT call wait() for 30 seconds.
During this time the terminated child becomes a zombie.
Use another terminal and run:
ps -o pid,ppid,state,stat,cmd -p 749
Child PID = 749
Child is terminating now.
Parent now calls waitpid() to collect child.
Zombie child has been collected.
```

Process Table – Zombie State:



Observations:

• The program creates a child process with parent PID 748 and child PID 749. The parent intentionally delays calling wait() for 30 seconds.

• The child process terminates immediately by calling exit(0). However, since the parent has not yet called wait() or waitpid(), the child process entry remains in the process table.

• A process table entry that remains after a child terminates is called a zombie process. The child execution has finished, but the OS retains the entry to preserve the child exit status for the parent.

• The ps output shows: "749 748 Z Z+ [P4_T2] <defunct>". The state column displays Z (zombie state) and the process is marked as <defunct>, confirming PID 749 is a zombie.

• The zombie process is not consuming CPU cycles; it is merely a process table entry waiting for the parent to retrieve its termination status.

• After 30 seconds elapse, the parent calls waitpid(pid, NULL, 0), which reaps the zombie child. The process table entry is removed and the parent receives the child termination status.

• Proper parent-child synchronization (calling wait() or waitpid() promptly after the child terminates) prevents accumulation of zombie processes and ensures clean process table management.